

Relazione Tecnica di Progetto: BitPub La Piattaforma per i Giochi Connessi

Indice

1. [Specifiche funzionali](#)
2. [Analisi tecnologica](#)
3. [Scelta dell'approccio tecnologico](#)
4. [Architettura del software](#)
5. [Implementazione](#)
6. [Validazione](#)

1. Specifiche funzionali

Obiettivi del Progetto e Problema Affrontato

Il progetto **BitPub** affronta la sfida di modernizzare e digitalizzare le classiche attività ludiche presenti in sale giochi e pub (calciobalilla, biliardo, freccette). L'obiettivo primario è prelevare le interazioni fisiche (es. un goal, un colpo al bersaglio) tramite sensori IoT, processarle in tempo reale per restituire feedback immediato ai giocatori in loco, e sincronizzarle sul Cloud per alimentare dinamiche tipiche degli *e-sports*, quali leaderboard globali, tornei e statistiche storiche.

Utenti del Sistema

L'applicativo gestisce nativamente il controllo degli accessi Role-Based (RBAC), distinguendo:

- **PLAYER:** L'utente finale. Può iscriversi ai tornei, consultare il catalogo giochi, accedere al proprio storico partite ed entrare in una *lobby* di gioco.
- **LOCALE_ADMIN:** Amministratore di una singola struttura fisica. Ha i permessi (verificati tramite un attributo `localeId` nel JWT) per registrare nuove macchine fisiche limitatamente al proprio locale.
- **PLATFORM_ADMIN:** Super-utente che gestisce l'intero catalogo giochi globale, configura i tornei ed ha visibilità su tutti i locali e i match.

Casi d'Uso e Funzionalità Offerte

Dal codice emergono le seguenti funzionalità di dominio implementate nel sistema:

1. **Matchmaking e Lobbies:** Il giocatore può scansionare una macchina in un pub e aprire una *lobby* virtuale (stato `WAITING_FOR_PLAYERS`). L'ingresso del secondo giocatore fa scattare la transizione allo stato `IN_PROGRESS`.
2. **Gameplay Real-Time:** I sensori fisici inviano *SensorEvents*. Il sistema rileva colpi, falli, score e chiusure di partita con validazione del turno (`currentTurnUserId`).
3. **Tornei E-Sports:** Gestione di tabelloni a eliminazione diretta. Il sistema (tramite `assertTournamentPairing`) convalida crittograficamente l'identità dei due giocatori connessi alla macchina fisica per impedire partite non autorizzate.
4. **Gestione Resilienza (Controllo Locale vs Internet):** Funzionalità cruciale del sistema. Il controllo del gameplay live (verifica del turno, incremento del punteggio) è **demandato in locale** al nodo *Edge* del pub, garantendo zero latenza. La comunicazione verso internet (statistiche, leaderboard) avviene in asincrono ed è tollerante ai guasti della rete.

2. Analisi tecnologica

Il progetto si basa su uno stack estremamente moderno e "cloud-native", diviso tra Frontend, Backend Cloud e Nodo Edge.

Backend e Cloud

- **Java 21 e Spring Boot 3.2.4:** Scelti per la stabilità, il vasto ecosistema e il supporto ai Virtual Threads (Project Loom) introdotti in Java 21, ideali per architetture ad alto throughput di I/O.
- **Spring Cloud Gateway:** Implementato nel gateway-service, funge da unico punto di accesso (API Gateway). Permette di centralizzare la logica di routing e terminare l'autenticazione JWT prima che la richiesta raggiunga i microservizi. L'alternativa (autenticazione decentralizzata su ogni servizio) avrebbe causato duplicazione di codice e problemi di accoppiamento.
- **PostgreSQL 15:** Database relazionale solido e ACID-compliant. Scelta naturale per mantenere la consistenza in transazioni complesse (come i pagamenti o i tabelloni dei tornei).

Frontend

- **React 19, TypeScript e TailwindCSS:** L'utilizzo di TS garantisce l'astrazione e la validazione dei DTO del backend, riducendo le regressioni. Tailwind permette un design sistematico senza file CSS sparsi.

IoT e Comunicazione

- **Eclipse Mosquitto 2.0 (MQTT)**: Broker leggero, open-source, standard de-facto nell'IoT. Gestisce i livelli di Qualità del Servizio (QoS), la conservazione degli stati live (Retained messages) e consuma pochissima RAM. L'alternativa AMQP (es. RabbitMQ) sarebbe risultata eccessivamente pesante da deployare su hardware Edge limitato (es. Raspberry Pi).

Infrastruttura

- **Docker e Docker Compose**: Per riprodurre fedelmente l'ambiente Edge-Cloud in locale.

3. Scelta dell'approccio tecnologico

Perché l'accoppiata Edge-Cloud e il Message Broker?

La problematica principale dei giochi connessi è la **connettività intermittente e la latenza**.

Se un giocatore fa "Goal" al calciobalilla:

- *Approccio REST puro (diretto al cloud)*: Se la connessione internet del pub cade per 2 secondi, il goal va perso o l'interfaccia si blocca. La latenza causa un disservizio.
- *Approccio WebSockets (diretto al cloud)*: Richiede una connessione TCP persistente aperta. Cade la connessione, cade il sistema.

La Soluzione Implementata:

BitPub sfrutta il pattern **Edge Computing accoppiato a MQTT**.

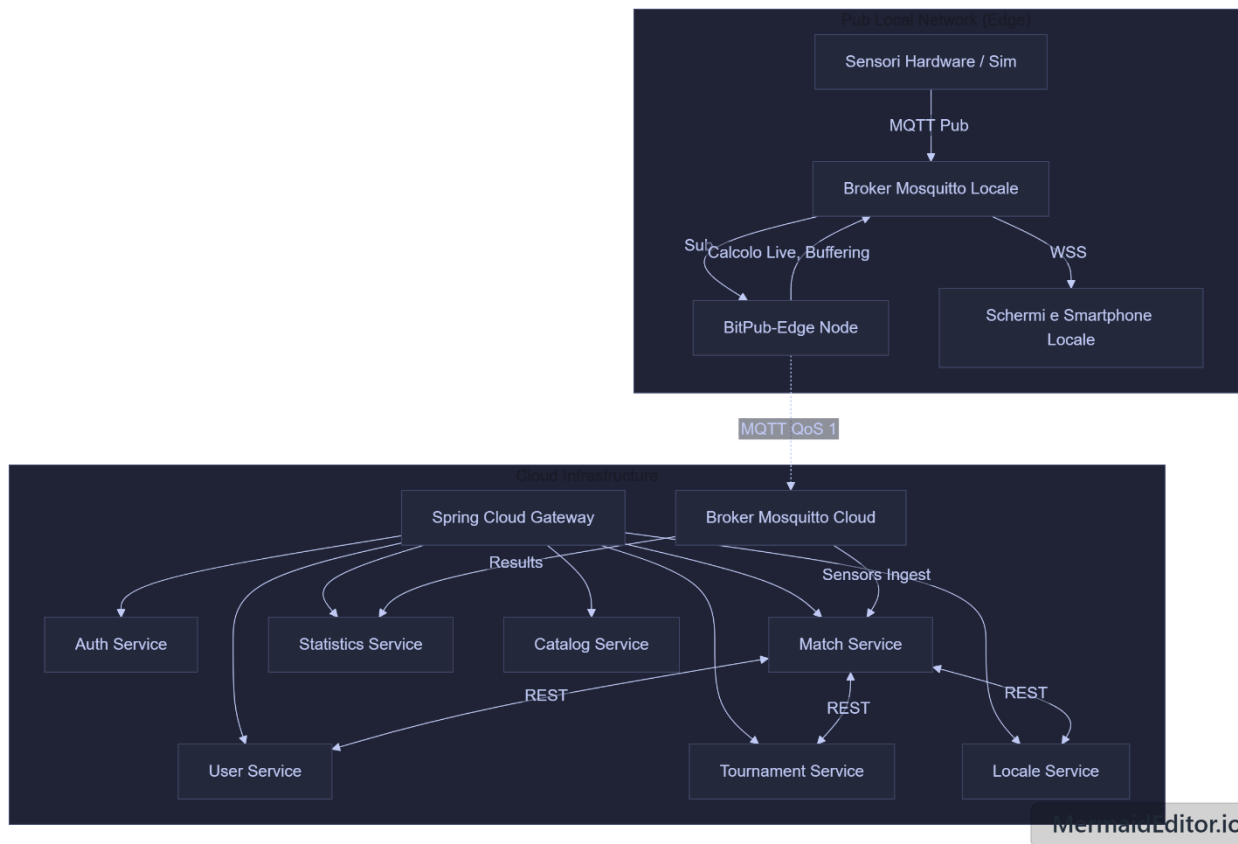
Nel pub risiede un broker MQTT (Mosquitto) locale e un'istanza Spring Boot (BitPub-Edge).

1. Il sensore pubblica su MQTT *locale*. L'applicazione Edge riceve istantaneamente, convalida il goal e pubblica il nuovo punteggio. I monitor nel pub sono aggiornati con latenza <5ms.
2. L'applicazione Edge, **in asincrono**, inoltra l'evento validato ai server in Cloud (bitpub/cloud/sensors/{id}/event).
3. Se internet cade, entra in gioco il MqttBufferService sviluppato ad hoc nell'Edge, che mette in coda in RAM gli eventi e li "spara" (flush) non appena la rete torna UP, sfruttando gli eventi MqttConnectionFailedEvent di Spring Integration.

Questa architettura isola il "business critico locale" (la partita) dal "business cloud" (le statistiche globali), risolvendo brillantemente il teorema CAP sacrificando una consistenza cloud immediata in favore della disponibilità locale (Eventual Consistency).

4. Architettura del software

Il sistema segue i pattern classici dei **Microservizi** con un API Gateway come Entrypoint.



Componenti e Flussi Dati

1. **Gateway Service:** Intercetta tutto il traffico HTTP frontend, estrae il token JWT, e compila header custom (X-User-Id, X-User-Role) instradando verso i servizi.
2. **Match Service:** Motore cloud. Quando un match passa allo stato IN_PROGRESS, invia un payload JSON completo via MQTT all'Edge (bitpub/edge/matches/{matchId}/sync). Questo permette all'Edge di gestire la partita anche se perde temporaneamente i contatti con i database centrali.
3. **Statistics e Tournament:** Moduli isolati. Consumano gli eventi "Partita Finita" dal Match Service in via asincrona (notifyStatisticsService via MQTT). Se le statistiche sono temporaneamente non disponibili, il Match Service continua a funzionare, applicando il principio dell'alta disponibilità.

5. Implementazione

L'implementazione tecnica fa forte uso di *Dependency Injection* (tramite le annotazioni di Lombok @RequiredArgsConstructor) e dei pattern architetturali di Domain-Driven Design (Entità -> Repository -> DTO -> Service -> Controller).

Backend: La Sicurezza a livello Gateway

La classe `JwtAuthenticationFilter` estende `AbstractGatewayFilterFactory`. Implementa il pattern *API Gateway Offloading*. Il token viene parsato qui usando `JwtUtils`. In caso di successo, si muta la richiesta originaria iniettando Headers sicuri:

```
ServerHttpRequest modifiedRequest = exchange.getRequest().mutate()

    .header("X-User-Id", userId)

    .header("X-User-Role", role)

    .build();
```

I microservizi a valle si "fidano" degli header e non necessitano di ricalcolare i certificati JWT, migliorando le performance complessive.

Backend: Gestione del Matchmaking e Concorrenza

Nel `MatchServiceImpl`, l'accesso in Lobby è gestito transazionalmente (`@Transactional`). Quando il secondo giocatore preme "Gioca", il metodo cerca se esiste una lobby `WAITING_FOR_PLAYERS`.

In caso di partita di torneo, viene effettuato un controllo rigoroso tramite API REST interna al `tournament-service` (`assertTournamentPairing`).

Un aspetto pregevole è l'uso di Spring `@RestClient` (il moderno rimpiazzo di `RestTemplate` introdotto in Spring Boot 3.2) per le comunicazioni sincrone cloud-to-cloud.

L'Idempotenza

L'IoT invia spesso duplicati. `MatchServiceImpl.processSensorEvent` previene bug gravissimi implementando un filtro di idempotenza all'ingresso:

```
if (sensorEventLogRepository.existsByEventId(eventId)) {

    log.warn("Event {} already processed, skipping.", eventId);

    return;

}
```

L'Edge Node: Gestione offline (`MqttBufferService.java`)

L'Edge Node possiede una logica sofisticata per la resilienza. Invece di affidarsi ciecamente alle code trasparenti di Paho (client MQTT), traccia la connessione con Spring Integration Events (`MqttConnectionFailedEvent`).

Usa una struttura dati *thread-safe* (`ConcurrentLinkedQueue`) per accodare in memoria gli eventi. Quando la connessione viene ristabilita (`MqttSubscribedEvent`), un blocco

synchronized esegue il flush dei messaggi verso il cloud in ordine sequenziale. Questo pattern è vitale nei sistemi cyber-fisici aziendali.

Frontend: React e MQTT Fallback

Il componente PlayFlow.tsx dimostra eccellenti pratiche di ingegneria.

L'hook useEffect apre una sottoscrizione via WebSockets:

```
const topic = `bitpub/match/${activeMatch.localeId}/${activeMatch.gameInstanceId}/state`;

const unsubscribe = notificationService.subscribe(topic, (payload: GameState) => {

  setGameState(payload);

  // ... transizione stato

});
```

Per tollerare il fatto che MQTT QoS 0 potrebbe far perdere messaggi ai client web, è stato implementato un fallback tramite polling. Un setInterval esegue una HTTP GET ogni 4 secondi (getMatch) come "ancora di salvezza" qualora il pacchetto MQTT di inizio partita andasse perso in una rete cellulare debole.

6. Validazione

Strategia di Testing

Il team ha preferito un approccio ai **Test E2E di Integrazione** piuttosto che concentrarsi unicamente su unit test sterili che spesso faticano a individuare errori architetturali.

Analisi dei test (MatchmakingE2ETest.java)

La classe di test fa uso della libreria **Testcontainers**. Quando il test viene lanciato:

1. Docker scarica e avvia dinamicamente un'istanza reale di postgres:15-alpine.
2. Il database viene "collegato" a Spring tramite @DynamicPropertySource.
3. Anziché lanciare il vero locale-service (che rallenterebbe eccessivamente i test), si adotta un pattern di Mocking tramite HttpServer nativo del JDK, simulando la risposta JSON per indicare che la macchina da gioco è "attiva".

Scenario Testato e Validato:

Il test secondPlayerJoiningTransitionsLobbyToInProgress verifica il flusso critico del sistema:

- L'utente A (alice) entra e il server risponde con WAITING_FOR_PLAYERS.
- L'utente B (bob) invia il join per lo stesso gameId.

- Il test valida il risultato atteso: l'identificativo partita generato (`lobby.getId()`) è identico (dimostrando l'aggregazione), lo stato del DTO restituito diviene `IN_PROGRESS` e la lista dei team raggiunge le 2 unità.

Considerazioni qualitative

Le pipeline di deployment sono ottimizzate. Lo script `scripts/rebuild_no_cache.ps1` avvia docker compose forzando le build, garantendo il clean run in fasi di sviluppo. I file YAML di configurazione espongono rigorosamente le variabili d'ambiente (es. URL al DB o ai broker) applicando correttamente il pattern del parametro d'ambiente "12 Factor App", garantendo al codice totale portabilità.